

Task 04 — Build Second Microservice & Implement Service Boundaries.

Step 1 — Create Order Service

Open:

order-service → pom.xml

Your dependencies should include these:

```
<dependencies>

  <!-- REST API -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-web</artifactId>

  </dependency>

  <!-- Database Operations -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-data-jpa</artifactId>

  </dependency>

  <!-- Request Validation -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-validation</artifactId>

  </dependency>

  <!-- MySQL Database -->

  <dependency>

    <groupId>com.mysql</groupId>
```

```
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <!-- Health Check -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
    <!-- Testing -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Create the Main Application Class

Go to:

src/main/java/com/example/orderservice

Create a Java class:

OrderServiceApplication.java

Code

```
package com.example.orderservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class OrderServiceApplication {
```

```
public static void main(String[] args) {  
    SpringApplication.run(OrderServiceApplication.class, args);  
}  
}
```

Step 2 — Create Order Entity

Create Package

Inside:

```
src/main/java/com/example/orderservice
```

Create a package:

```
entity
```

Create a Java class:

```
Order.java
```

Order.java

```
package com.example.orderservice.entity;  
  
import jakarta.persistence.*;  
  
import java.math.BigDecimal;  
  
import java.time.LocalDateTime;  
  
@Entity  
@Table(name = "orders")  
  
public class Order {  
  
    @Id  
  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
  
    private Long id;  
  
    private Long userId;  
  
    private String productName;  
  
    private Integer quantity;
```

```
private BigDecimal amount;

private String status;

private LocalDateTime createdAt;

// Default constructor

public Order() {

}

// Getters and Setters

public Long getId() {

    return id;

}

public void setId(Long id) {

    this.id = id;

}

public Long getUserId() {

    return userId;

}

public void setUserId(Long userId) {

    this.userId = userId;

}

public String getProductName() {

    return productName;

}

public void setProductName(String productName) {

    this.productName = productName;

}
```

```
public Integer getQuantity() {  
    return quantity;  
}  
  
public void setQuantity(Integer quantity) {  
    this.quantity = quantity;  
}  
  
public BigDecimal getAmount() {  
    return amount;  
}  
  
public void setAmount(BigDecimal amount) {  
    this.amount = amount;  
}  
  
public String getStatus() {  
    return status;  
}  
  
public void setStatus(String status) {  
    this.status = status;  
}  
  
public LocalDateTime getCreatedAt() {  
    return createdAt;  
}  
  
public void setCreatedAt(LocalDateTime createdAt) {  
    this.createdAt = createdAt;  
}  
}
```

Explanation

`@Entity`

Tells Spring Boot:

This class represents a database table.

`@Table(name = "orders")`

The table name will be:

orders

`@Id`

This is the primary key.

`@GeneratedValue(strategy = GenerationType.IDENTITY)`

The database automatically generates the ID.

Example:

1

2

3

4

```
private Long userId;
```

Stores the ID of the user who placed the order.

Important: We are storing `userId`, not creating a Java relationship with the User entity. This keeps the services independent.

```
private BigDecimal amount;
```

Used for money values.

```
private LocalDateTime createdAt;
```

Stores when the order was created.

Step 3 — Create Order Repository

A **Repository** is responsible for database operations.

Create Package

repository

Create:

OrderRepository.java

OrderRepository.java

```
package com.example.orderservice.repository;

import com.example.orderservice.entity.Order;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface OrderRepository extends JpaRepository<Order, Long> {

    List<Order> findByUserId(Long userId);

}
```

Explanation

JpaRepository<Order, Long>

Means:

Order → Entity

Long → Primary Key Type

Spring automatically provides:

save()

findById()

findAll()

deleteById()

existsById()

Custom Method

```
List<Order> findByUserId(Long userId);
```

This means:

Find all orders belonging to a particular user.

Example:

```
GET /api/orders/user/101
```

Returns:

```
[  
  {  
    "id": 1,  
    "userId": 101,  
    "productName": "Laptop",  
    "quantity": 1,  
    "amount": 60000,  
    "status": "CREATED"  
  }  
]
```

Step 4 — Create Order DTOs

DTO means **Data Transfer Object**.

It is used to transfer data between:

Client → Controller → Service

CreateOrderRequest

Create package:

```
dto
```

Create:

```
CreateOrderRequest.java
```

```
package com.example.orderservice.dto;

import jakarta.validation.constraints.*;

import java.math.BigDecimal;

public class CreateOrderRequest {

    @NotNull(message = "User ID is required")
    private Long userId;

    @NotBlank(message = "Product name is required")
    private String productName;

    @NotNull(message = "Quantity is required")
    @Min(value = 1, message = "Quantity must be at least 1")
    private Integer quantity;

    @NotNull(message = "Amount is required")
    @DecimalMin(value = "0.01", message = "Amount must be greater than 0")
    private BigDecimal amount;

    public CreateOrderRequest() {
    }

    public Long getUserId() {
        return userId;
    }

    public void setUserId(Long userId) {
        this.userId = userId;
    }

    public String getProductName() {
        return productName;
    }

    public void setProductName(String productName) {
```

```
        this.productName = productName;
    }

    public Integer getQuantity() {
        return quantity;
    }

    public void setQuantity(Integer quantity) {
        this.quantity = quantity;
    }

    public BigDecimal getAmount() {
        return amount;
    }

    public void setAmount(BigDecimal amount) {
        this.amount = amount;
    }
}
```

Example Request

```
{
  "userId": 101,
  "productName": "Laptop",
  "quantity": 2,
  "amount": 60000
}
```

Validation

@NotNull

Value cannot be null.

@NotBlank

String cannot be empty.

```
@Min(1)
```

Quantity must be at least 1.

```
@DecimalMin("0.01")
```

Amount must be greater than 0.

UpdateOrderRequest

Create:

```
UpdateOrderRequest.java
```

```
package com.example.orderservice.dto;
```

```
import jakarta.validation.constraints.*;
```

```
import java.math.BigDecimal;
```

```
public class UpdateOrderRequest {
```

```
    @NotBlank(message = "Product name is required")
```

```
    private String productName;
```

```
    @NotNull(message = "Quantity is required")
```

```
    @Min(value = 1, message = "Quantity must be at least 1")
```

```
    private Integer quantity;
```

```
    @NotNull(message = "Amount is required")
```

```
    @DecimalMin(value = "0.01", message = "Amount must be greater than 0")
```

```
    private BigDecimal amount;
```

```
    public UpdateOrderRequest() {
```

```
}
```

```
    public String getProductName() {
```

```
        return productName;
```

```
}
```

```
    public void setProductName(String productName) {
```

```
        this.productName = productName;
    }

    public Integer getQuantity() {
        return quantity;
    }

    public void setQuantity(Integer quantity) {
        this.quantity = quantity;
    }

    public BigDecimal getAmount() {
        return amount;
    }

    public void setAmount(BigDecimal amount) {
        this.amount = amount;
    }
}
```

Example

```
{
    "productName": "Laptop",
    "quantity": 3,
    "amount": 90000
}
```

OrderResponse

Create: OrderResponse.java

```
package com.example.orderservice.dto;

import java.math.BigDecimal;
import java.time.LocalDateTime;
```

```
public class OrderResponse {  
    private Long id;  
    private Long userId;  
    private String productName;  
    private Integer quantity;  
    private BigDecimal amount;  
    private String status;  
    private LocalDateTime createdAt;  
    public OrderResponse() {  
    }  
    public Long getId() {  
        return id;  
    }  
    public void setId(Long id) {  
        this.id = id;  
    }  
    public Long getUserId() {  
        return userId;  
    }  
    public void setUserId(Long userId) {  
        this.userId = userId;  
    }  
    public String getProductName() {  
        return productName;  
    }  
    public void setProductName(String productName) {  
        this.productName = productName;  
    }  
}
```

```
public Integer getQuantity() {  
    return quantity;  
}  
public void setQuantity(Integer quantity) {  
    this.quantity = quantity;  
}  
public BigDecimal getAmount() {  
    return amount;  
}  
public void setAmount(BigDecimal amount) {  
    this.amount = amount;  
}  
public String getStatus() {  
    return status;  
}  
public void setStatus(String status) {  
    this.status = status;  
}  
public LocalDateTime getCreatedAt() {  
    return createdAt;  
}  
public void setCreatedAt(LocalDateTime createdAt) {  
    this.createdAt = createdAt;  
}  
}
```

The database entity is not directly returned to the client.

Instead: Entity → OrderResponse → Client

This is a clean service boundary.

Step 5 — Create Order Service Layer

Now we create the business logic.

Flow

Controller



Service



Repository



Database

Create package:

service

Create:

OrderService.java

OrderService.java

```
package com.example.orderservice.service;

import com.example.orderservice.dto.CreateOrderRequest;
import com.example.orderservice.dto.OrderResponse;
import com.example.orderservice.dto.UpdateOrderRequest;
import com.example.orderservice.entity.Order;
import com.example.orderservice.repository.OrderRepository;
import org.springframework.stereotype.Service;
import java.time.LocalDateTime;
import java.util.List;

@Service
public class OrderService {

    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
```

```

        this.orderRepository = orderRepository;
    }

    // Create Order
    public OrderResponse createOrder(CreateOrderRequest request) {
        Order order = new Order();
        order.setUserId(request.getUserId());
        order.setProductName(request.getProductName());
        order.setQuantity(request.getQuantity());
        order.setAmount(request.getAmount());
        order.setStatus("CREATED");
        order.setCreatedAt(LocalDateTime.now());
        Order savedOrder = orderRepository.save(order);
        return convertToResponse(savedOrder);
    }

    // Get All Orders
    public List<OrderResponse> getAllOrders() {
        return orderRepository.findAll()
            .stream()
            .map(this::convertToResponse)
            .toList();
    }

    // Get Order By ID
    public OrderResponse getOrderById(Long id) {
        Order order = orderRepository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("Order not found with id: " + id));
        return convertToResponse(order);
    }
}

```

```

// Get Orders By User ID
public List<OrderResponse> getOrdersByUserId(Long userId) {
    return orderRepository.findByUserId(userId)
        .stream()
        .map(this::convertToResponse)
        .toList();
}

// Update Order
public OrderResponse updateOrder(
    Long id,
    UpdateOrderRequest request) {
    Order order = orderRepository.findById(id)
        .orElseThrow(() ->
            new RuntimeException("Order not found with id: " + id));
    order.setProductName(request.getProductName());
    order.setQuantity(request.getQuantity());
    order.setAmount(request.getAmount());
    Order updatedOrder = orderRepository.save(order);
    return convertToResponse(updatedOrder);
}

// Delete Order
public void deleteOrder(Long id) {
    if (!orderRepository.existsById(id)) {
        throw new RuntimeException("Order not found with id: " + id);
    }
    orderRepository.deleteById(id);
}

// Convert Entity to DTO

```

```

private OrderResponse convertToResponse(Order order) {
    OrderResponse response = new OrderResponse();
    response.setId(order.getId());
    response.setUserId(order.getUserId());
    response.setProductName(order.getProductName());
    response.setQuantity(order.getQuantity());
    response.setAmount(order.getAmount());
    response.setStatus(order.getStatus());
    response.setCreatedAt(order.getCreatedAt());
    return response;
}
}

```

Step 6 — Create Order APIs

Method	API	Purpose
POST	/api/v1/orders	Create a new order
GET	/api/v1/orders/{id}	Get one order
GET	/api/v1/orders/user/{userId}	Get orders of a user
PUT	/api/v1/orders/{id}/status	Update order status
DELETE	/api/v1/orders/{id}	Cancel an order

Step 7 — Define Service Ownership

User Service owns:

User ID
 Name
 Email
 Password
 Phone

Order Service owns:

Order ID

User ID

Product Name

Quantity

Amount

Status

Created At

Order Service must NOT directly access User Service database.

Service Ownership

User Service

Owns:

- User data
- User registration
- User profile
- User authentication

Database:

- user_db

Order Service

Owns:

- Order data
- Order creation
- Order status
- Order cancellation

Database:

- order_db

Service Boundary

Order Service stores only `userId`.

Order Service must not directly access the User Service database.

The User Service owns user-related data, and the Order Service owns order-related data. The Order Service stores only the `userId` to identify the customer. It does not directly access the User Service database, which keeps both services independent.

Step 8 — Configure Separate Databases

Create the databases

Open **MySQL Workbench** or MySQL Command Line.

Run:

```
CREATE DATABASE user_db;
```

```
CREATE DATABASE order_db;
```

Check:

```
SHOW DATABASES;
```

Expected:

```
user_db
```

```
order_db
```

Configure User Service

Open your **User Service** project in IntelliJ IDEA or Eclipse.

Go to:

```
src
```

```
└─ main
```

```
    └─ resources
```

```
        └─ application.properties
```

Add:

```
spring.application.name=user-service
```

```
server.port=8081
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/user_db
```

```
spring.datasource.username=root
```

```
spring.datasource.password=YOUR_MYSQL_PASSWORD
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Configure Order Service

Open your **Order Service** project.

Go to:

src

└─ main

└─ resources

└─ application.properties

Add: spring.application.name=order-service

server.port=8082

spring.datasource.url=jdbc:mysql://localhost:3306/order_db

spring.datasource.username=root

spring.datasource.password=YOUR_MYSQL_PASSWORD

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true

Step 9 — Run Both Services

Run User Service

Open User Service in IntelliJ.

Find the main class:

@SpringBootApplication

```
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            UserServiceApplication.class,
            args
        )
    }
}
```

```
    );  
}  
}
```

Expected console:

Tomcat started on port 8081

Started UserServiceApplication

Run Order Service

Open Order Service.

Find:

@SpringBootApplication

```
public class OrderServiceApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(  
            OrderServiceApplication.class,  
            args  
        );  
    }  
}
```

Expected console:

Tomcat started on port 8082

Started OrderServiceApplication

Correct:

User Service → 8081

Order Service → 8082

Step 10 — Test Independently

Test User Service

Open **Postman**.

Create User

Method: POST

URL: http://localhost:8081/users

Body → raw → JSON

```
{  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

Expected response:

```
{  
  "id": 1,  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

Get All Users

Method: GET

URL: http://localhost:8081/users

Expected:

```
[  
  {  
    "id": 1,  
    "name": "Ravi",  
    "email": "ravi@gmail.com"  
  }  
]
```

Test Order Service

Create Order

Method: POST

URL: http://localhost:8082/orders

Body:

```
{  
  "userId": 1,  
  "productName": "Laptop",  
  "amount": 50000  
}
```

Expected response:

```
{  
  "id": 1,  
  "userId": 1,  
  "productName": "Laptop",  
  "amount": 50000  
}
```

Get All Orders

Method: GET

URL: http://localhost:8082/orders

Expected:

```
[  
  {  
    "id": 1,  
    "userId": 1,  
    "productName": "Laptop",  
    "amount": 50000  
  }  
]
```

Deliverables

User Service

Submit your complete User Service project.

Example:

user-service/

├── pom.xml

└── src/

 └── main/

 ├── java/

 └── com/example/userservice/

 ├── UserServiceApplication.java

 ├── controller/

 └── UserController.java

 ├── entity/

 └── User.java

 └── repository/

 └── UserRepository.java

 |

 └── resources/

 └── application.properties

2. Order Service Submit your complete Order Service project.

order-service/

├── pom.xml

└── src/

 └── main/

 ├── java/

 └── com/example/orderservice/

 ├── OrderServiceApplication.java

```
|    |— controller/
|    |   └─ OrderController.java
|    |— entity/
|    |   └─ Order.java
|    └─ repository/
|        └─ OrderRepository.java
|
└─ resources/
    └─ application.properties
```

Separate Databases

Show:

user_db

order_db

Separate Repositories

This means:

UserRepository

OrderRepository

Each service has its own repository.

5. REST APIs

POST /users

GET /users

POST /orders

GET /orders

Service Boundary Document

Create a simple document explaining:

User Service

Responsible for:

- Creating users
- Managing users
- Retrieving users

Database:

- user_db

Port:

- 8081

Order Service

Responsible for:

- Creating orders
- Managing orders
- Retrieving orders

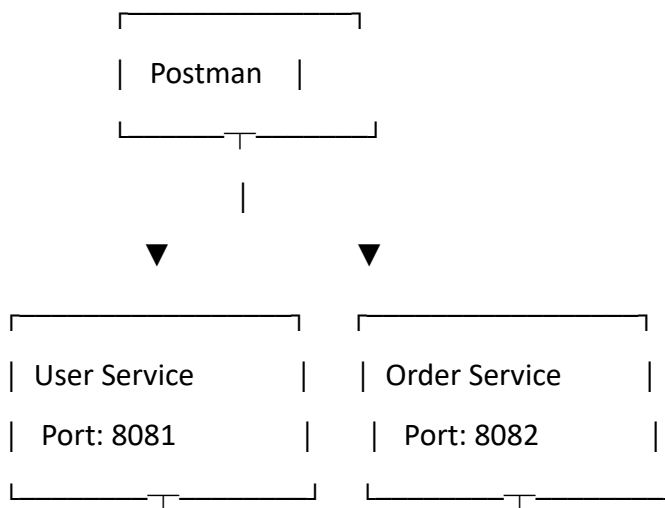
Database:

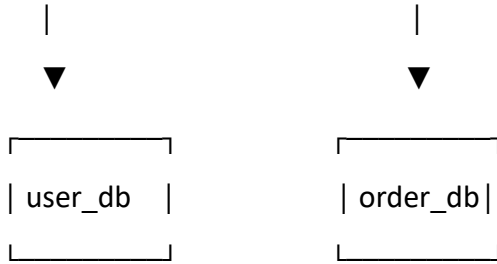
- order_db

Port:

- 8082

Architecture Diagram





Postman Collection

A **Postman Collection** is a saved group of API requests.

Instead of submitting only screenshots, you can export the collection and share it.

Example collection

Microservices Collection

